A Summer Internship Project Report on

"Emergency Blood Connector: A Full-Stack Web-Based Platform for Real-Time Blood Donor-Patient Matching and Emergency Request Management"

Submitted in partial fulfilment of the Requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

By

Maralakshikari Sreeshwan
24EG105Q47

Under the Guidance of

_____

Department of Computer Science & Engineering
ANURAG UNIVERSITY
Venkatapur, Ghatkesar,
Medchal-Malkajgiri District, Hyderabad – 500088,
Telangana, India

Academic Year: 2026-2027

ANURAG UNIVERSITY

Venkatapur, Ghatkesar, Medchal-Malkajgiri District, Hyderabad – 500088, Telangana, India

Department of Computer Science & Engineering

CERTIFICATE

I, Maralakshikari Sreeshwan, bearing hall ticket number, **24EG105Q47**, hereby declare that the project report entitled **"Emergency Blood Connector: A Full-Stack Web-Based Platform for Real-Time Blood Donor-Patient Matching and Emergency Request Management"**, Department of Computer Science & Engineering, Anurag University, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering**.

This is a record of bonafide work carried out by me and the results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Signature of The Guide

Maralakshikari Sreeshwan

_____

24EG105Q47

ACKNOWLEDGMENT

I would like to express my sincere thanks and deep sense of gratitude to project supervisor _____, for his constant encouragement and inspiring guidance without which this project could not have been completed. His critical reviews and constructive comments improved our grasp of the subject and steered to the fruitful completion of the work. His patience, guidance and encouragement made this project possible.

I would like to express my sincere gratitude to Dr. Archana Mantri, Vice Chancellor, Anurag University and Dr. P. Bhaskara Reddy, Registrar, Anurag University for their encouragement and support.

I would like to express my special thanks to Dr. V. Vijaya Kumar, Dean School of Engineering, Anurag University, for his encouragement and timely support in our B.Tech program.

I would like to acknowledge my sincere gratitude for the support extended by Dr. G. Vishnu Murthy, Dean, Department of Computer Science Engineering, Anurag University. I also express my deep sense of gratitude to Dr. P. Ravinder Rao, academic coordinator, whose research expertise and commitment to the highest standards continuously motivated me during the crucial stage of my project work.

Maralakshikari Sreeshwan
24EG105Q47

DECLARATION

I hereby declare that the project entitled **"Emergency Blood Connector: A Full-Stack Web-Based Platform for Real-Time Blood Donor-Patient Matching and Emergency Request Management"** submitted to the Department of Computer Science and Engineering is an original work carried out by me under the guidance of _____. The work presented in this report has not been submitted elsewhere for the award of any degree.

Maralakshikari Sreeshwan
24EG105Q47

TABLE OF CONTENTS

TOPICS **PAGE NO.**

Acknowledgement.....3
Declaration..... 4
1. Introduction..... 6
2. Problem Statement.....8
3. Objectives.....9
4. Scope of the Project.....10
5. Existing System.....12
6. Proposed System..... 15
7. Software & Hardware Requirements.....19
8. System Design.....21
9. Flowchart & Algorithm.....23
10. Modules Description.....26
11. Implementation.....28
12. Results / Output Screens.....31
13. Advantages & Limitations..... 40
14. Future Enhancements..... 42
15. Conclusion.....43
16. References..... 44
17. Source Code.....45

(Page numbers above update automatically in Microsoft Word — if they show as “1”, select all text with Ctrl+A then press F9, or right-click and choose “Update Field.”)

INTRODUCTION

Access to safe and timely blood remains one of the most critical yet fragile links in emergency healthcare, particularly in a country as populous and geographically diverse as India. Every year, thousands of patients face life-threatening delays not because compatible donors do not exist, but because there is no efficient, real-time mechanism to connect a patient in need with an available, eligible donor nearby. Hospitals and blood banks maintain their own limited donor registries, but these are rarely interoperable, rarely searchable by the general public, and almost never account for real-time donor availability, blood group compatibility, or the urgency of the underlying medical situation.

Traditionally, the burden of finding a donor during an emergency falls on the patient's family or friends, who resort to phone calls, WhatsApp broadcasts, and social media appeals. This process is slow, unstructured, and emotionally taxing at a time when speed matters most. There is no way to verify donor eligibility (such as whether a donor is still within their mandatory rest period after a previous donation), no way to prioritise the most critical cases over routine ones, and no way to track whether a donor who agreed to help actually completed the donation. The absence of a centralised, structured system means that the same effort is repeated for every new request, with no institutional memory of past donors, their reliability, or their availability.

The project "Emergency Blood Connector" (EBC) is a full-stack MERN (MongoDB, Express.js, React, Node.js) web application built to directly address this gap. It provides a single platform where patients or their attendants (Requesters) can raise a structured, verifiable blood request; where willing donors (Donors) can discover and respond to compatible, nearby requests; and where platform administrators (Admins) can oversee the entire donor-request ecosystem. The system introduces a severity-based alert mechanism (GREEN, YELLOW, RED, BLACK) that automatically prioritises the most critical requests, a two-step donation confirmation workflow that prevents false claims of donation, and a gamified reward system — points, donor levels, and badges — that encourages repeat donation and long-term donor engagement.

Built using React 19 on the frontend and Express.js 5 with Mongoose 9 on the backend, and deployed using Vercel and Render with MongoDB Atlas as the cloud database, Emergency Blood Connector is designed to be lightweight, fast, and accessible from any device with a browser. The platform's role-based architecture (DONOR, REQUESTER, ADMIN) ensures that every stakeholder in the blood donation ecosystem — from the patient's family to the platform administrator — has a purpose-built interface suited to their needs, while a shared, consistent data layer keeps every request, donation, and notification synchronised in real time.

What distinguishes Emergency Blood Connector from a simple contact directory is that it treats blood donation as a trackable, ongoing relationship rather than a one-time transaction. Every donor accumulates a verifiable history of confirmed donations, every request carries a transparent, auditable lifecycle from creation through fulfilment, and every stakeholder — donor, requester, and administrator — can see the state of the system that matters to them without needing to make a single phone call. This report documents the problem the project addresses, the system design and algorithms underpinning it, its implementation, and the results achieved, before concluding with its limitations and a roadmap for future enhancement.

The project was undertaken as a summer internship exercise with the explicit goal of building a genuinely usable, deployable system rather than a purely academic prototype — which is why every feature described in this report, from blood-group compatibility filtering to the donation confirmation workflow, exists as working, tested code in the live deployment referenced in the Source Code section, rather than as a proposed or partially implemented feature.

PROBLEM STATEMENT

In emergency medical situations requiring blood transfusion, patients and their families in India continue to rely on ad-hoc, manual methods — phone calls, personal networks, and social media appeals — to locate compatible blood donors. This approach suffers from several critical shortcomings that directly affect patient outcomes. There is no centralised registry of donors that can be filtered by blood group, geographic location, and real-time availability, which means requesters must contact dozens of potential donors individually, most of whom may be ineligible, unavailable, or too far away to help in time.

Existing hospital-level donor lists are rarely digitised, are not accessible to the public, and provide no mechanism for a donor to see which requests are compatible with their blood group without manually cross-referencing compatibility charts. There is also no standard way to prioritise requests by medical urgency — a routine, planned transfusion request is treated with the same visibility as a life-threatening emergency, causing critical cases to be lost in the noise of ordinary appeals.

Furthermore, once a donor agrees to help, there is no formal record of whether the donation actually took place, whether the donor followed the mandatory post-donation rest period before donating again, or whether the request was ultimately fulfilled. This absence of verification opens the system to both accidental double-booking of donors and outright false claims, while also making it impossible to recognise or reward donors who consistently show up. Emergency Blood Connector is designed to solve exactly this problem: providing a structured, verifiable, and prioritised digital platform that closes the gap between blood availability and blood need.

OBJECTIVES

1. To develop a full-stack, role-based web application that connects blood donors, requesters (patients/attendants), and administrators on a single centralised platform, replacing ad-hoc phone-and-social-media based donor search.
2. To implement a secure authentication and authorization system using JSON Web Tokens (JWT) delivered via HTTP-only cookies, combined with bcrypt password hashing and strict role-based access control (RBAC) for the DONOR, REQUESTER, and ADMIN roles.
3. To design an automatic blood-group compatibility engine that filters open blood requests for each donor based on medically accurate donor-to-recipient compatibility rules, so that donors only see requests they are actually eligible to fulfil.
4. To implement a severity-based alert and priority scoring system (GREEN, YELLOW, RED, BLACK) that automatically ranks open requests by medical urgency and units required, ensuring the most critical cases surface first.
5. To build a two-step donation verification workflow — donor self-reporting followed by requester confirmation — so that reward points, donor levels, and the mandatory donation cooldown period are only applied once a donation is independently verified.
6. To design a gamified donor engagement system consisting of a points economy, six-tier donor levels (Iron through Diamond), milestone badges, and a public leaderboard, to encourage sustained, repeat blood donation.
7. To provide administrators with a dedicated dashboard for monitoring users, requests, donations, and platform-wide statistics, including blood group distribution, alert-level distribution, and state-wise user distribution.
8. To develop a responsive, mobile-friendly user interface using React 19, Tailwind CSS 4, and Redux Toolkit 2 that works seamlessly across desktop and mobile devices for all three user roles.
9. To deploy the complete application on cloud infrastructure — the frontend on Vercel and the backend on Render, backed by MongoDB Atlas — ensuring the platform is accessible from anywhere with an internet connection.

SCOPE OF THE PROJECT

The "Emergency Blood Connector" application encompasses the following functionalities:

Donor and Requester Registration and Login: Users register by providing their first name, last name, email, phone number, password, blood group, and state. Every account is assigned a role of DONOR, REQUESTER, or ADMIN. Authentication is handled through JWT tokens issued at login and verified via an Authorization Bearer header on every protected request, with passwords secured using bcrypt hashing. Registered users may optionally upload a profile image, which is processed through Multer and stored via Cloudinary.

Blood Request Creation and Management: Requesters can create a structured blood request containing patient name, age, gender, required blood group, number of units required, hospital name and address, state, contact person, contact number, an alert level (GREEN/YELLOW/RED/BLACK), and a required-by date. Each request is automatically assigned a unique, human-readable request number in the format REQ-YYYY-NNNNN, a calculated priority score, and an automatic expiry date proportional to its urgency. Requesters can view, edit, close, or soft-delete their own open requests, and can track fulfilment progress through a dedicated Requester Dashboard.

Donor Discovery and Compatibility Matching: Donors are shown only those open requests that are both blood-group compatible with their own blood group and located within their registered state, sorted by priority score. A donor may accept a compatible request, mark themselves as having completed the donation, and track the full lifecycle of every request they have engaged with, from acceptance through to confirmed donation, via their Donor Dashboard.

Verified Donation Workflow: When a donor marks a donation as complete, the system creates a PENDING donation record rather than immediately crediting the donor. Only once the requester (who can directly verify the donation occurred at the hospital) confirms it does the system award reward points, update the donor's level and badges, apply the 90-day donation cooldown, mark the donor as temporarily unavailable, and increment the request's fulfilled unit count — automatically marking the request as FULFILLED once all required units are met.

Gamification — Points, Levels, Badges, and Leaderboard: Every confirmed donation awards points scaled to the alert level of the request it fulfilled. Accumulated points determine the donor's tier — Iron, Bronze, Silver, Gold, Platinum, or Diamond — and unlock

milestone badges based on donation count and total points. A public leaderboard and a personal rank endpoint let donors see how they compare to the wider donor community.

Notification System: The platform generates in-app notifications for key events — request creation, a donor accepting a request, a donation awaiting confirmation, a donation being confirmed, and new badges earned — each visible through a dedicated notifications page with an unread-count indicator and mark-as-read functionality.

Administrative Oversight: Administrators can view and manage all registered users (including activating or deactivating accounts), inspect donors and requesters separately, view all blood requests segmented by status (open, fulfilled, deleted), force-close any request, and access an analytics dashboard summarising platform-wide statistics including user counts, request counts, donation counts, blood group distribution, alert-level distribution, and state-wise user distribution.

Responsive Design: The interface, built with Tailwind CSS 4 utility classes, adapts fluidly between a desktop layout with full navigation and a mobile-optimised layout, ensuring the platform is usable both by donors responding on the move and requesters managing a request from a hospital waiting room.

Out of Scope:

– SMS and WhatsApp based donor notifications (in-app notifications only in the current version)

– OTP-based mobile number verification during registration

– Password reset via email (currently requires an authenticated change-password flow)

– Real-time (WebSocket-based) live notification delivery — notifications are currently fetched on demand

– Hospital-side verification workflow for isHospitalVerified requests

– Native mobile application for Android or iOS

– Payment gateway or monetary transaction handling of any kind

– Multi-language support

EXISTING SYSTEM

1. Manual and Social-Media-Based Outreach (Traditional)

The most common method used by patients' families in an emergency is to personally call known contacts, post appeals in WhatsApp and Facebook groups, and circulate the request through word of mouth, hoping someone in their network matches the required blood group and is willing to travel to the hospital in time.

– No way to filter appeals by blood group compatibility or genuine proximity

– No verification that a responding "donor" is medically eligible or truthful about availability

– Appeals are repeated from scratch for every new emergency, with no reusable donor history

– Highly dependent on the reach of the requester's personal and social network

– No way to track whether the donation was actually completed

2. Hospital and Blood Bank Registers

Individual hospitals and blood banks maintain their own donor registers, often on paper or in disconnected spreadsheets, recording past donors who can be contacted when stock runs low. These registers are useful locally but are not interoperable across institutions.

– Donor data is siloed within a single hospital or blood bank, invisible to other institutions

– No public-facing search interface for patients or attendants

– Donor availability and eligibility (rest-period compliance) is rarely tracked in real time

– No systematic way to prioritise requests by medical urgency

3. eRaktKosh — Government of India Blood Bank Portal

Developed by the Centre for Development of Advanced Computing (C-DAC) under the Ministry of Health and Family Welfare's National Health Mission, eRaktKosh is India's official centralised platform for blood bank information, connecting thousands of registered blood banks nationwide. It allows citizens to search real-time blood stock availability by state, district, blood group, and component, locate nearby blood banks and donation camps, and register as a voluntary donor.

– Designed primarily around blood bank inventory and camp management rather than direct peer-to-peer emergency requester-to-donor matching

– No gamification, reward, or repeat-engagement mechanism for individual donors

– No automatic severity-based prioritisation of individual patient emergency requests

– Workflow is oriented around institutional blood banks rather than a lightweight requester-donor interaction

4. Friends2Support — Voluntary Blood Donor Database

Friends2Support is a long-running, India-based non-profit online platform that maintains one of the country's largest voluntary blood donor databases, allowing patients or attendants to directly search and contact registered donors by blood group and location without an intermediary.

– Donor-requester contact happens off-platform (phone calls), with no in-platform request lifecycle, acceptance flow, or donation confirmation

– No structured verification step to confirm a donation actually took place

– No alert-level or priority scoring to distinguish a routine request from a life-threatening one

– No reward, badge, or level system to encourage sustained donor engagement

5. State and District Blood Helpline Services

Several Indian state health departments and district administrations operate dedicated blood-request helpline numbers, staffed by call-centre operators who manually try to match an incoming request against whatever donor contacts or blood-bank stock information they have on hand at that moment.

– Entirely manual, human-mediated matching with no automated compatibility or priority logic

– Response quality and speed depend heavily on the individual operator and the freshness of the data available to them

– No self-service option for a requester or donor to interact with the system directly

– No persistent digital record of a donor's history or reliability across multiple requests

Common Limitations Across Existing Solutions:

– No unified platform that combines real-time compatibility matching, urgency-based prioritisation, and a verified donation lifecycle in one system

– Little to no gamification or long-term incentive structure to retain repeat donors

– Limited or no automatic enforcement of donor eligibility rules such as the mandatory post-donation rest period

– Donation completion is rarely independently verified by the party who benefited from it

– Administrative visibility into platform-wide trends (blood group shortages, alert-level distribution, regional demand) is limited or entirely absent for the end user

– No consistent, cross-institutional way to verify a donor's eligibility history or recent donation record, leaving eligibility checks dependent on the donor's own memory or word

– Data privacy, consent, and security practices vary widely between platforms, with little standardisation or oversight of how a donor's personal and medical information is stored and shared

PROPOSED SYSTEM

Emergency Blood Connector proposes a unified, free, role-based web platform that combines everything the existing solutions offer only in fragments — real-time compatibility matching, urgency-based prioritisation, a verified donation lifecycle, and long-term donor engagement through gamification — into a single, cohesive system. The application follows a clean client-server architecture with a clear separation of concerns between the frontend (a React 19 single-page application), the backend (an Express.js 5 REST API server), and the database (MongoDB accessed through Mongoose 9 ODM), ensuring the codebase remains modular, testable, and independently deployable.

Frontend Architecture:

The frontend is built using React 19 with Vite 8 as the build tool and Tailwind CSS 4 for utility-first styling. The application is organised by role into distinct page groups — public pages (Home, Open Requests, Request Detail, Notifications), authentication pages (Login, Register), donor pages (Donor Dashboard, Donation History, Badges, Achievements, Leaderboard), requester pages (Requester Dashboard, Create Request, My Requests, Edit Request), profile pages (Profile, Edit Profile, Change Password), and admin pages (Admin Dashboard, Admin Users, Admin Requests, Admin Statistics). Global authentication state and unread-notification state are managed through Redux Toolkit 2 slices (authSlice, notifSlice), while forms across the application are built with React Hook Form 7 for lightweight, performant validation. Route access is enforced client-side through a ProtectedRoute component that checks both authentication status and role permissions before rendering role-restricted pages.

Backend Architecture:

The backend is developed using Express.js 5 with ECMAScript Modules (import/export syntax) and is organised into five domain-specific API routers — CommonAPI (authentication and profile), RequestAPI (blood request lifecycle), DonorAPI (donor actions and gamification), NotificationAPI (in-app notifications), and AdminAPI (platform administration) — each mounted under its own base path (/auth, /request-api, /donor-api, /notification-api, /admin-api). Business logic that needs to be reused across routes — blood compatibility checking, points calculation, level calculation, badge calculation, priority scoring, expiry-date calculation, and donation-cooldown calculation — is factored into small, pure utility functions under a dedicated utils/ directory, keeping the route handlers focused on request validation, database access, and

response formatting. Every Mongoose schema enforces timestamps, disables the version key, and uses strict:"throw" mode so that any attempt to persist an undeclared field raises an error rather than silently succeeding.

Severity-Based Alert and Priority Engine:

Every blood request is tagged with one of four alert levels — GREEN, YELLOW, RED, or BLACK — representing increasing degrees of medical urgency. On creation, the system automatically computes a numeric priority score by combining a base score for the alert level with the number of units required, and automatically calculates an expiry date proportionate to the urgency (a GREEN request remains open for seven days, while a BLACK request expires in just one day if unfulfilled). The public Open Requests feed and each donor's personalised matched-requests feed are both sorted by this priority score, ensuring the most critical, time-sensitive cases surface first without requiring any manual triage.

Blood Compatibility Engine:

Rather than requiring donors to manually cross-reference blood-type compatibility charts, the backend implements a compatibility lookup utility encoding standard donor-to-recipient rules (for example, an O-negative donor is compatible with every blood group, while an AB-positive recipient can accept from any group). This utility is applied automatically whenever a donor views open requests, ensuring they only ever see requests they are medically eligible to fulfil, and is re-validated server-side at the moment a donor attempts to accept a request.

Two-Step Verified Donation Workflow:

To prevent both accidental double-booking and false donation claims, the donation process is split into two independently-triggered steps. First, a donor accepts a compatible, open request, after which they may mark the donation as complete — which creates a PENDING donation record rather than immediately rewarding the donor. Second, the requester (who is present at the hospital and can directly verify that the donation took place) confirms the donation. Only at confirmation does the system award points, recompute the donor's level and badges, apply the 90-day donation cooldown, mark the donor temporarily unavailable, and increment the request's fulfilled-unit count, automatically transitioning the request to FULFILLED once all required units are met.

Gamification and Donor Engagement Engine:

Every confirmed donation awards reward points scaled to the urgency of the request it fulfilled, directly incentivising donors to respond to the most critical cases. Accumulated points place each donor into one of six tiers — Iron, Bronze, Silver, Gold, Platinum, and Diamond — and unlock milestone badges tied to both total donation count and total points earned. A public leaderboard, a top-donors endpoint, and a personal-rank endpoint let donors track their standing within the wider donor community, turning what is otherwise a purely altruistic act into a recognised, trackable achievement.

Notification System:

The platform raises an in-app notification for every significant lifecycle event — a new request being created, a donor accepting a request, a donation awaiting the requester's confirmation, a donation being confirmed, and a new badge being earned — each persisted to the database and surfaced through a dedicated notifications page with an unread-count badge and mark-as-read control, ensuring every stakeholder stays informed without needing to repeatedly check request status manually.

Administrative Analytics Engine:

Beyond day-to-day moderation, the admin dashboard aggregates platform-wide metrics on demand — total, available, and unavailable donor counts; total, open, fulfilled, closed, and deleted request counts; total confirmed donations; blood-group distribution across the donor base; alert-level distribution across active requests; and state-wise user distribution. Rather than requiring a separate reporting pipeline, these statistics are computed directly from the same MongoDB collections that power the rest of the application, using Mongoose aggregation queries evaluated at request time, keeping the numbers administrators see always current.

Key Differentiators from Existing Solutions:

– A single, unified request-to-donation lifecycle rather than fragmented inventory portals or contact-only donor databases

– Automatic, medically accurate blood-compatibility filtering applied to every donor's request feed

– Automatic severity-based priority scoring and urgency-proportional expiry, with no manual triage required

– A two-step, independently-verified donation workflow that prevents both double-booking and false donation claims

– A full gamification layer — points, levels, badges, leaderboard — designed to retain donors well beyond a single emergency

– A dedicated administrative analytics layer for platform-wide blood-group, alert-level, and state-wise distribution

– Completely free, open, and built on modern, actively maintained web technologies

– A single, mobile-responsive codebase that serves donors, requesters, and administrators from the same deployment, rather than separate portals, apps, or institutional logins

SOFTWARE AND HARDWARE REQUIREMENTS

Table 1: Software and Hardware Requirements

Software	Version	Purpose
Operating System	Windows 10/11, Linux, or macOS	Development environment
Node.js	18+	JavaScript runtime for the backend
npm	9+	Package manager
MongoDB	MongoDB Atlas (Cloud)	Database server
VS Code	Latest	Code editor
Google Chrome	Latest	Testing browser
React	19.2.6	Frontend UI library
Vite	8.0.12	Frontend build tool
Tailwind CSS	4.3.1	CSS framework
Redux Toolkit	2.12.0	Global state management
React Router	7.17.0	Client-side routing
React Hook Form	7.79.0	Form state and validation
Axios	1.17.0	HTTP client
Express.js	5.1.0	Backend web framework
Mongoose	9.0.0	MongoDB ODM
JSON Web Token	9.0.2	Authentication
bcryptjs	3.0.2	Password hashing

Software	Version	Purpose
Multer	2.2.0	Multipart file upload handling
Cloudinary	2.10.0	Cloud image storage for profile pictures
cookie-parser	1.4.7	HTTP-only cookie parsing
Vercel	Latest	Frontend hosting
Render	Latest	Backend hosting

Hardware Requirements:

– Processor: Intel Core i3 or equivalent (minimum), Intel Core i5 or higher (recommended)

– RAM: 4 GB minimum, 8 GB recommended

– Storage: 10 GB free disk space

– Internet: Broadband connection with a minimum of 2 Mbps speed

– Display: 1366 x 768 resolution minimum

– Input: Keyboard and mouse, or a touchscreen device for mobile access

SYSTEM DESIGN

The system follows a three-tier client-server architecture: a React 19 single-page application forms the presentation tier, an Express.js 5 REST API forms the application tier, and MongoDB — accessed through Mongoose ODM — forms the data tier. Every backend endpoint is namespaced under a domain-specific base path (/auth, /request-api, /donor-api, /notification-api, /admin-api) and, with the exception of a small number of intentionally public read endpoints (open requests, leaderboard, top donors, request health checks), is protected by JWT-based authentication and role-based authorization.

Entity-Relationship Overview:

```
User (1) ──→ (N) BloodRequest     [ requester creates many requests ]
User (1) ──→ (N) Donation         [ donor completes many donations ]
User (1) ──→ (N) Notification     [ user receives many notifications ]
BloodRequest (1) ──→ (N) Donation    [ request fulfilled by many donations ]
```

Collection Field Overview:

Table 2: Database Collection Field Overview

User	BloodRequest	Donation	Notification
firstName, last Name	requestNumber	donorId (ref: User)	userId (ref: User)
email (unique)	patientName, patientAge, patientGender	bloodRequestId (ref: Blood Request)	title
password (hashed)	bloodGroup, units Required, unitsFulfilled	requestNumber	message
phoneNumber (unique)	hospitalName, hospital Address, state	alertLevel	type (enum)

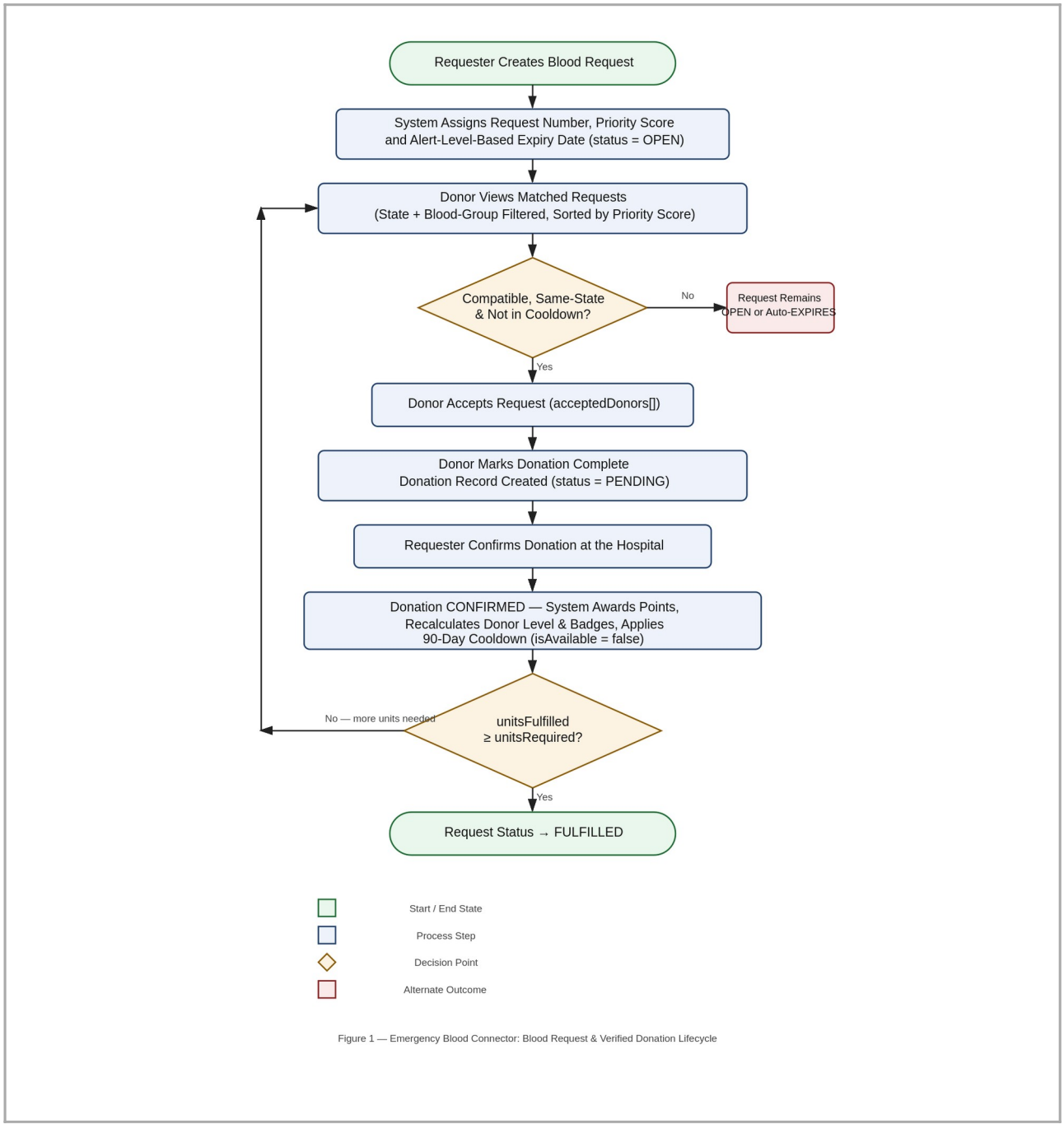
User	BloodRequest	Donation	Notification
bloodGroup, state	contactPerson, contact Number	points Awarded, units Donated	isRead
role (DONOR / REQUESTER / ADMIN)	requestCreatedBy (ref: User)	donationDate	createdAt
isAvailable	alertLevel, status, priority Score	nextEligible DonationDate	
donations Count, total Points, donor Level, badges	acceptedDonors[], pending Confirmation[], completed Donors[]	status (PENDING / CONFIRMED / REJECTED)	
lastDonation Date, next Eligible DonationDate	isHospitalVerified	isVerified	

The BloodRequestModel additionally maintains three donor-tracking arrays that together encode the full lifecycle of a request: acceptedDonors (donors who have accepted but not yet donated), pendingConfirmation (donors who have marked their donation complete and are awaiting requester confirmation, cross-referenced to their DonationModel record), and completedDonors (donors whose donation has been independently confirmed). A compound index on { role, isAvailable, state, bloodGroup } on the User collection, and an index on { status, priorityScore } on the BloodRequest collection, keep the most frequent queries — donor matching and the priority-sorted open-requests feed — efficient as the dataset grows.

FLOWCHART AND ALGORITHM

The diagram below traces the complete lifecycle of a blood request from creation through fulfilment, including the compatibility and priority checks a donor must satisfy before acting on it, and the two independent confirmation steps that gate every reward, level, and cooldown update. The pseudocode for each stage is detailed further below the diagram.

Figure 1: Blood Request and Verified Donation Lifecycle



Algorithm 1 — Blood Request Creation and Prioritisation

> INPUT: patientDetails, bloodGroup, unitsRequired, hospitalDetails, alertLevel

```
requestNumber ← "REQ-" + currentYear + "-" + zeroPadded(sequentialCount)
priorityScore ← alertLevelScore[alertLevel] + unitsRequired
  where alertLevelScore = { GREEN:10, YELLOW:30, RED:60, BLACK:100 }
expiresAt ← now + expiryDays[alertLevel]
  where expiryDays = { GREEN:7, YELLOW:5, RED:3, BLACK:1 }
SAVE BloodRequest{ status:"OPEN", unitsFulfilled:0, priorityScore, expiresAt, ... }
NOTIFY requester → "Blood Request Created"
```

Algorithm 2 — Donor Matching and Compatibility Filter

```
INPUT: donor (bloodGroup, state)
openRequests ← FIND BloodRequest WHERE status="OPEN" AND state = donor.state
            SORTED BY priorityScore DESC, createdAt DESC
matchedRequests ← FILTER openRequests WHERE canDonate(donor.bloodGroup,
request.bloodGroup)
RETURN matchedRequests
```

Algorithm 3 — Two-Step Verified Donation Workflow

```
STEP 1 — ACCEPT REQUEST (Donor):
  IF donor.isAvailable = false → REJECT "in cooldown period"
  IF donor HAS a PENDING donation on any request → REJECT
  IF request.status ≠ "OPEN" OR unitsFulfilled ≥ unitsRequired → REJECT
  IF request.state ≠ donor.state OR NOT canDonate(donor, request) → REJECT
  IF donor already in request.acceptedDonors → REJECT
  ELSE: request.acceptedDonors.push({donorId, acceptedAt})
        NOTIFY requester → "Request Accepted"
```

```
STEP 2 — MARK DONATION COMPLETE (Donor):
  IF donor NOT IN request.acceptedDonors → REJECT
  IF donor already IN completedDonors OR pendingConfirmation → REJECT
  ELSE: pointsAwarded ← calculatePoints(request.alertLevel)
        nextEligibleDate ← donationDate + 90 days
        CREATE Donation{ status:"PENDING", pointsAwarded, nextEligibleDate }
        MOVE donor: acceptedDonors → pendingConfirmation
        NOTIFY requester → "Donation Awaiting Confirmation"
```

```
STEP 3 — CONFIRM DONATION (Requester / Admin):
  donation.status ← "CONFIRMED";  donation.isVerified ← true
  donor.totalPoints += donation.pointsAwarded
  donor.donationsCount += 1
  donor.donorLevel ← calculateLevel(donor.totalPoints)
  donor.badges ← calculateBadges(donor.donationsCount, donor.totalPoints)
  donor.isAvailable ← false;  donor.nextEligibleDonationDate ← donation.nextEligibleDate
  MOVE donor: pendingConfirmation → completedDonors
  request.unitsFulfilled += 1
```

```
  IF request.unitsFulfilled ≥ request.unitsRequired → request.status ← "FULFILLED"
  NOTIFY donor → "Donation Confirmed" (+ "New Badge Earned" for each newly unlocked
badge)
```

Algorithm 4 — Automatic Request Expiry

```
ON EVERY read of open-requests / my-requests:
  UPDATE ALL BloodRequest WHERE status="OPEN" AND expiresAt < NOW
  SET status = "EXPIRED"
```

Algorithm 5 — Administrative Oversight and Analytics

```
ACTIVATE / DEACTIVATE USER (Admin):
  target ← FIND User BY userId
  IF target NOT FOUND → REJECT "User Not Found"
  target.isUserActive ← requestedStatus;  SAVE target
```

```
FORCE CLOSE REQUEST (Admin, any ownership):
  request ← FIND BloodRequest BY requestNumber
  IF request.status IN ("FULFILLED","CLOSED","DELETED") → REJECT
  request.status ← "CLOSED";  SAVE request
```

```
COMPUTE PLATFORM STATISTICS:
  bloodGroupDistribution ← GROUP User BY bloodGroup, COUNT
  alertLevelDistribution ← GROUP BloodRequest BY alertLevel, COUNT
  stateDistribution      ← GROUP User BY state, COUNT
  RETURN { totals, bloodGroupDistribution, alertLevelDistribution, stateDistribution }
```

MODULES DESCRIPTION

1. Authentication & Profile Module:

Handles registration (with an optional profile image processed through Multer and Cloudinary), login, session verification, logout, profile editing, and password change, exposed through the CommonAPI router (/auth). Passwords are hashed with bcryptjs before storage, and a successful login issues a JWT signed with the user's id and role, which the client attaches as a Bearer token on every subsequent protected request. A dedicated verifyToken middleware factory accepts a list of allowed roles and rejects the request with a 401 or 403 status if the token is missing, expired, or the user's role is not permitted.

2. Blood Request Management Module:

Exposed through RequestAPI (/request-api), this module lets requesters create, view, edit, close, and soft-delete blood requests. On creation, it automatically generates a unique request number, computes the priority score and expiry date from the chosen alert level, and initialises the donor-tracking arrays. The open-requests and my-requests endpoints both perform a lazy auto-expiry sweep — flipping any overdue OPEN request to EXPIRED — before returning results, and a requester dashboard endpoint aggregates total, open, and fulfilled request counts alongside total units required versus fulfilled.

3. Donor Matching & Action Module:

Exposed through DonorAPI (/donor-api), this is the module donors interact with most. It computes a donor's matched-requests feed (state and blood-group filtered, priority sorted), tracks a donor's accepted and completed requests, and implements the two-step accept-request / complete-donation actions with full eligibility validation — cooldown status, existing pending donations, request status, state match, and blood-group compatibility — enforced server-side before any state change is persisted.

4. Donation Confirmation Module:

A dedicated confirm-donation endpoint within RequestAPI allows only the requester who created the request (or an admin) to verify a donor's pending donation. This single operation atomically updates the DonationModel record to CONFIRMED, recalculates the donor's totalPoints, donorLevel, and badges, applies the 90-day cooldown by setting isAvailable to false

and computing nextEligibleDonationDate, moves the donor from pendingConfirmation to completedDonors on the request, increments unitsFulfilled, and auto-transitions the request to FULFILLED once fully satisfied — followed by notifications for the confirmed donation and any newly unlocked badges.

5. Gamification Module:

A set of small, pure calculation utilities — calculatePoints (alert-level based reward), calculateLevel (six-tier threshold mapping), and calculateBadges (donation-count and points-based milestones) — are invoked whenever a donation is confirmed, keeping the reward logic centralised, testable, and consistent regardless of which route triggers it. DonorAPI additionally exposes read endpoints for a donor's own badge and achievement summary, a public leaderboard, a top-3 donors list, and a personal-rank lookup computed by sorting all donors by total points and donation count.

6. Notification Module:

Exposed through NotificationAPI (/notification-api), this module persists an in-app notification for every significant lifecycle event via a shared createNotification utility, invoked from the request-creation, request-acceptance, donation-marking, donation-confirmation, and badge-earning code paths. Users can fetch their own notification history sorted by recency, retrieve an unread count for a badge indicator, and mark individual notifications as read — all scoped strictly to the authenticated user's own userId.

7. Administrative Module:

Exposed through AdminAPI (/admin-api) and restricted entirely to the ADMIN role, this module provides full visibility into the platform: listing all users (with separate donor and requester views), inspecting a specific user's full profile, activating or deactivating accounts, listing all requests segmented by status (all, open, fulfilled, deleted), inspecting a specific request's full detail, and force-closing any request regardless of ownership. A dashboard endpoint returns high-level platform counts, while a dedicated statistics endpoint aggregates blood-group distribution, alert-level distribution, and state-wise user distribution for administrative analysis.

IMPLEMENTATION

The project is implemented as a monorepo with two independently deployable packages — backend/ and frontend/ — each with its own package.json and dependency tree. Both packages use ECMAScript Module syntax throughout, and the backend layers its logic as Routes (in APIs/) calling shared utility functions (in utils/) that operate on Mongoose models (in models/), keeping business rules such as point calculation or expiry-date computation reusable and independently verifiable rather than duplicated inline across route handlers.

Backend Structure:

The backend entry point, server.js, initialises the Express application, configures CORS with a filtered list of allowed origins (dropping any undefined FRONTEND_URL value so a missing environment variable never silently blocks every production request), enables JSON body parsing and cookie parsing, and mounts all five API routers under their respective base paths. The database connection is established through Mongoose's connect() method using the DB_URL environment variable, and the HTTP server only begins listening once that connection succeeds — preventing the application from serving traffic against a database it cannot reach. A catch-all 404 handler returns a structured JSON error for invalid paths, and a global four-argument error-handling middleware normalises Multer errors, file-filter errors, Mongoose ValidationError and CastError, and MongoDB duplicate-key (11000) errors into consistent JSON error responses, logging the full stack trace server-side while withholding internal error detail from the client in production.

The models/ directory contains four Mongoose schema definitions: UserModel, BloodRequestModel, DonationModel, and NotificationModel — each configured with timestamps enabled, the version key disabled, and strict:"throw" mode so that any attempt to save an undeclared field throws rather than silently succeeding.

The middlewares/ directory contains the single verifyToken factory function used across every protected route, and the config/ directory holds the Multer memory-storage configuration and the Cloudinary upload helper used for profile image handling.

Frontend Structure:

The frontend entry point, main.jsx, renders the React application wrapped in a Redux Provider and BrowserRouter. The root App.jsx defines every application route, guarding role-restricted paths behind a ProtectedRoute component that reads authentication and role state from the Redux

authSlice. Pages are organised by role into public/, auth/, donor/, requester/, profile/, and admin/ folders, while a shared components/ directory holds cross-cutting UI pieces such as Navbar, RequestCard, and ProtectedRoute. All outbound HTTP calls are routed through a single configured Axios instance (api/axiosInstance.js) that centralises the base URL and credential handling, and a shared styles/common.js module centralises reusable Tailwind class strings so visual styling stays consistent across every page without duplicating class strings inline.

Development Approach:

The backend was built incrementally over a structured, module-by-module schedule rather than all at once, which kept each layer independently testable before the next was built on top of it. Development began with requirements analysis, database design, and API/folder structure planning, followed by the authentication module (registration, login, JWT issuance, cookie handling), the blood request module (creation, request numbering, priority scoring, expiry calculation), the donor acceptance flow, the donation completion and cooldown system, the rewards and gamification engine (points, levels, badges, leaderboard), the notification module, the remaining request-management endpoints (edit/close/delete/dashboard), the donor dashboard, the full administrative module (user and request management, activation controls), and finally platform-wide analytics and statistics — with a dedicated pass at the end devoted specifically to authentication, request, donation, notification, dashboard, admin, statistics, and authorization testing before the backend was marked feature-complete.

Testing Approach:

In the absence of an automated test suite, each API module was verified manually using dedicated HTTP request files (common.http, request.http, donor.http, notification.http) written in the ###Label convention supported by REST client tooling. Each file groups every endpoint of its module as an independently runnable request — pre-filled with representative JSON payloads and a placeholder Bearer token — allowing every route, including role-restricted and negative-path scenarios (invalid tokens, wrong role, cooldown violations, duplicate acceptance, already-confirmed donations), to be exercised repeatedly during development without relying on the frontend being complete. This endpoint-level verification was carried out for every module before the corresponding frontend pages were integrated against it.

Deployment:

The frontend is built with npm run build, producing a static dist/ bundle deployed to Vercel with automatic CI/CD triggered from the GitHub repository. The backend is deployed to Render, listening on the port supplied via the PORT environment variable. MongoDB Atlas hosts the database as a fully managed cloud cluster, with the connection string, JWT secret key, and frontend origin URL supplied as environment variables on the Render dashboard, and the Cloudinary credentials supplied similarly for profile-image uploads.

Sample API Requests and Responses:

The following representative request/response pairs illustrate the contract exposed by the core endpoints of the system.

```
POST /auth/register
{ "firstName":"Anita", "lastName":"Rao", "email":"anita@example.com",
  "password":"********", "phoneNumber":"9001000001",
  "bloodGroup":"O+", "state":"Telangana", "role":"DONOR" }
→ 201 { "message":"User created successfully", "payload": { ... } }
```

```
POST /request-api/create-request      (Role: REQUESTER, ADMIN)
{ "patientName":"Ramesh Kumar", "patientAge":40, "patientGender":"MALE",
  "bloodGroup":"O+", "unitsRequired":2, "hospitalName":"Apollo Hospitals",
  "hospitalAddress":"Jubilee Hills, Hyderabad", "state":"Telangana",
  "contactPerson":"Dr. Sharma", "contactNumber":"9999999999",
  "alertLevel":"RED", "requiredByDate":"2026-07-10" }
→ 201 { "message":"Blood Request Created Successfully",
       "payload": { "requestNumber":"REQ-2026-00011", "priorityScore":62, ... } }
```

```
PUT /donor-api/accept-request        (Role: DONOR)
{ "requestNumber":"REQ-2026-00011" }
→ 200 { "message":"Donation Request Accepted Successfully", "payload": { ... } }
```

```
PATCH /request-api/confirm-donation   (Role: REQUESTER, ADMIN)
{ "requestNumber":"REQ-2026-00011", "donorId":"665f1c..." }
→ 200 { "message":"Donation confirmed successfully",
       "payload": { "request": { "status":"FULFILLED", "unitsFulfilled":2 },
                    "donation": { "status":"CONFIRMED", "pointsAwarded":25 } } }
```

```
GET /donor-api/leaderboard            (Public)
→ 200 { "payload": [ { "rank":1, "donorName":"Anita Rao",
                       "donorLevel":"Gold", "totalPoints":140 }, ... ] }
```

RESULTS / OUTPUT SCREENS

The following screens are captured directly from the deployed Emergency Blood Connector application, illustrating the key interfaces available to donors, requesters, and administrators. Each figure is captioned and numbered for easy reference.

Figure 2: Home Page (Public Landing)

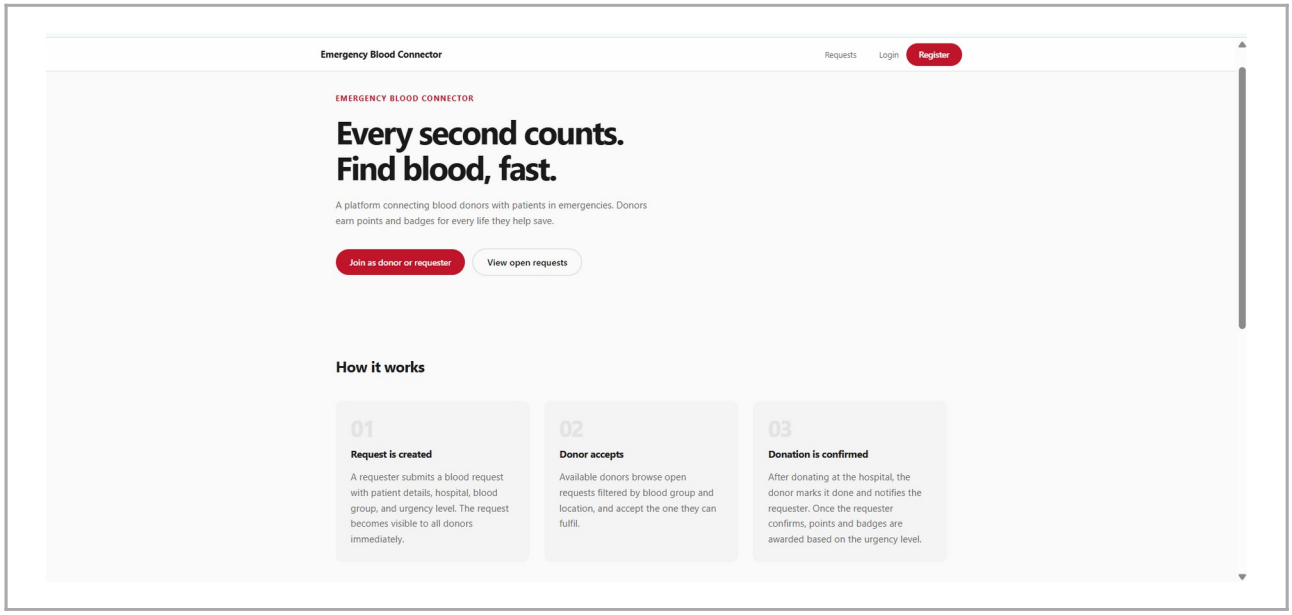


Figure 3: Login Page

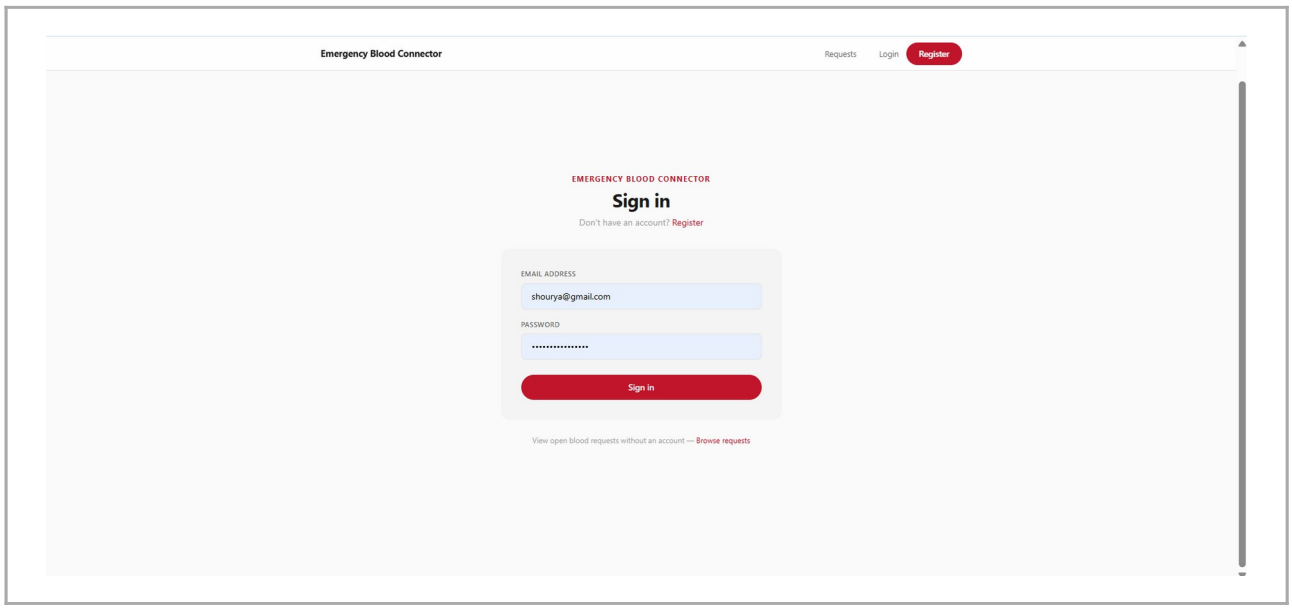


Figure 4: Register Page

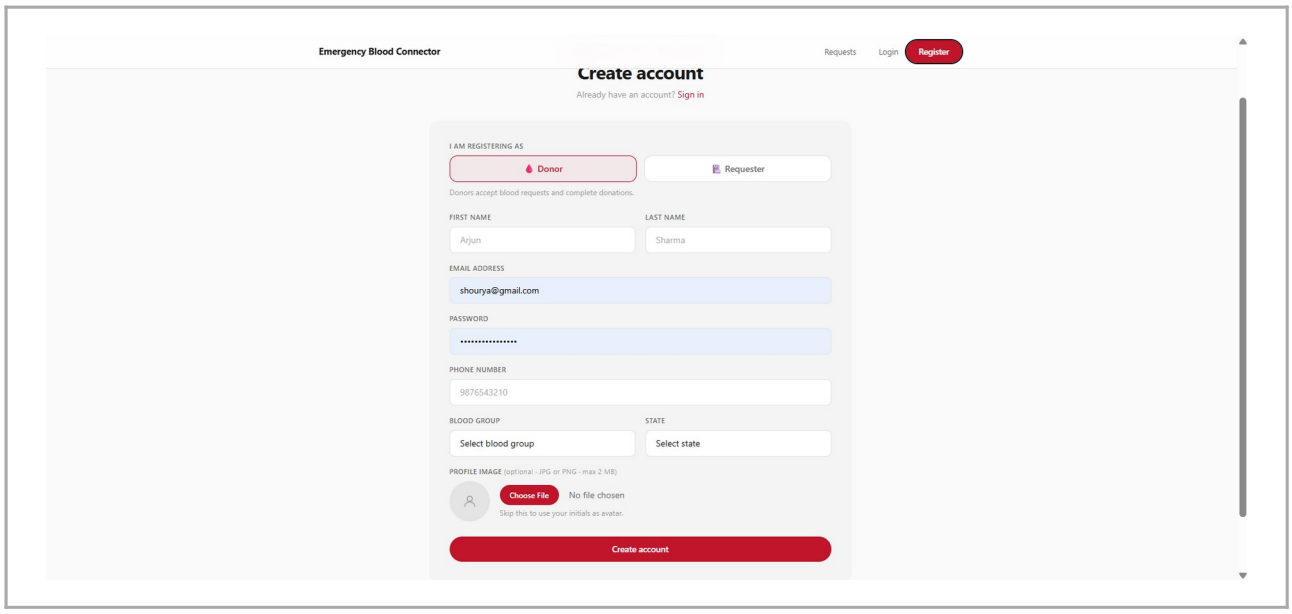


Figure 5: Donor Dashboard

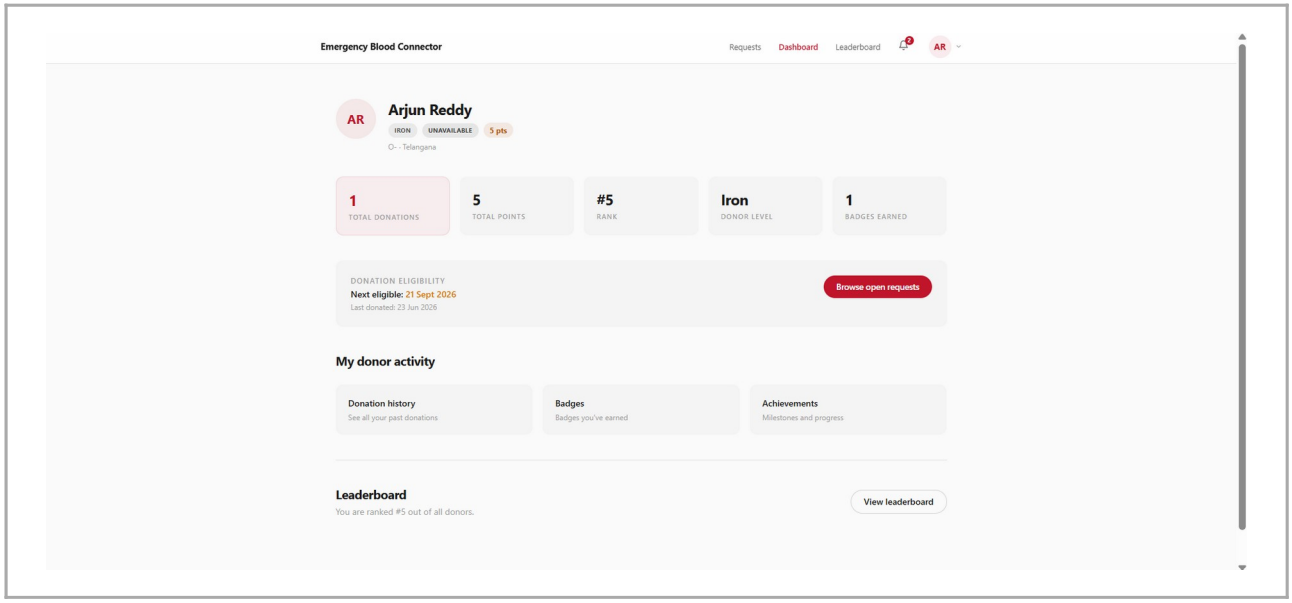


Figure 6: Requester Dashboard

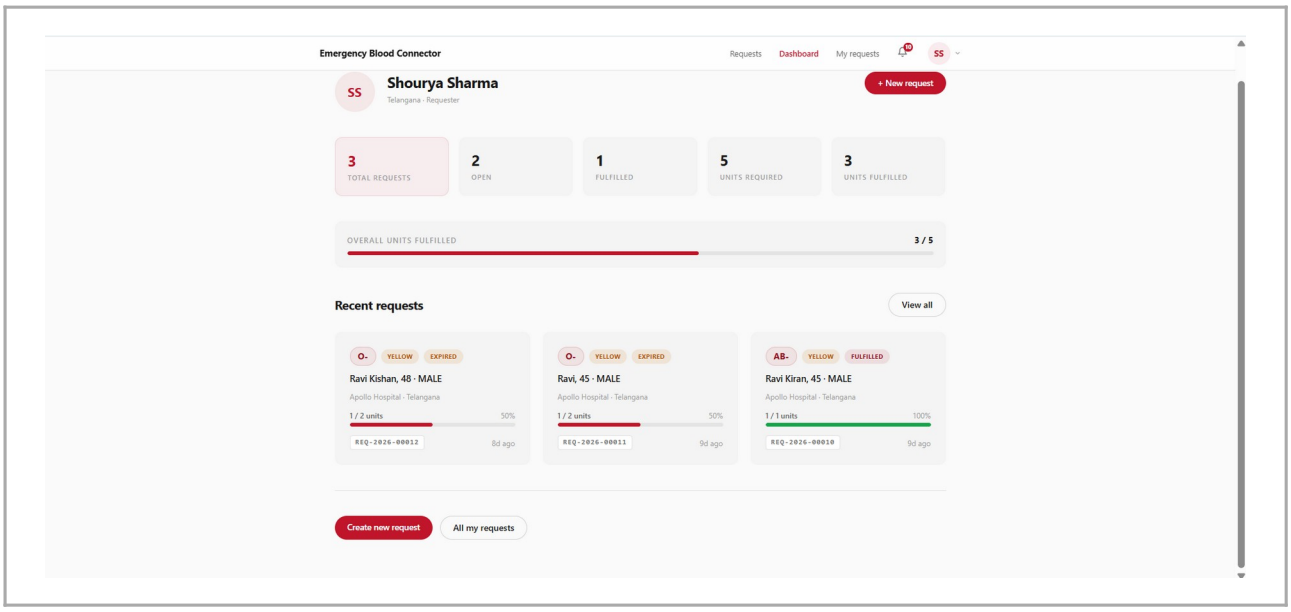


Figure 7: Open Blood Requests Feed

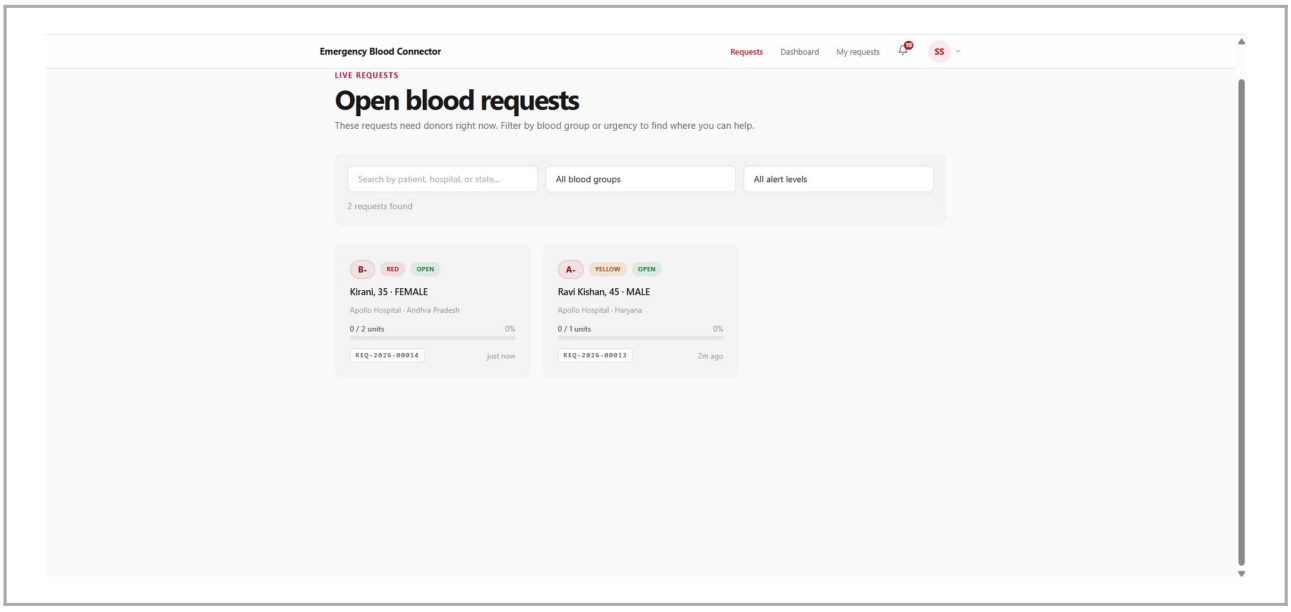


Figure 8: Create Blood Request Form

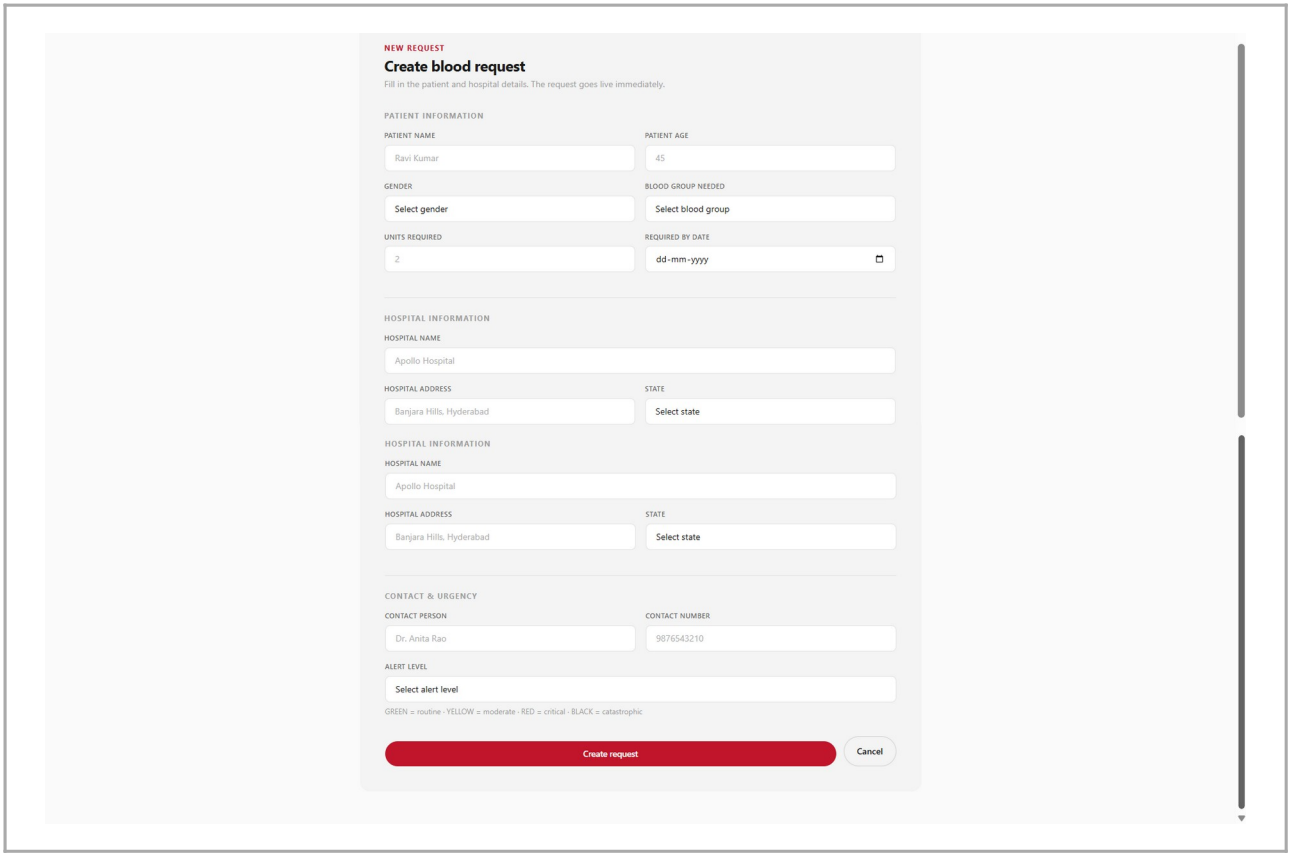


Figure 9: Request Detail Page

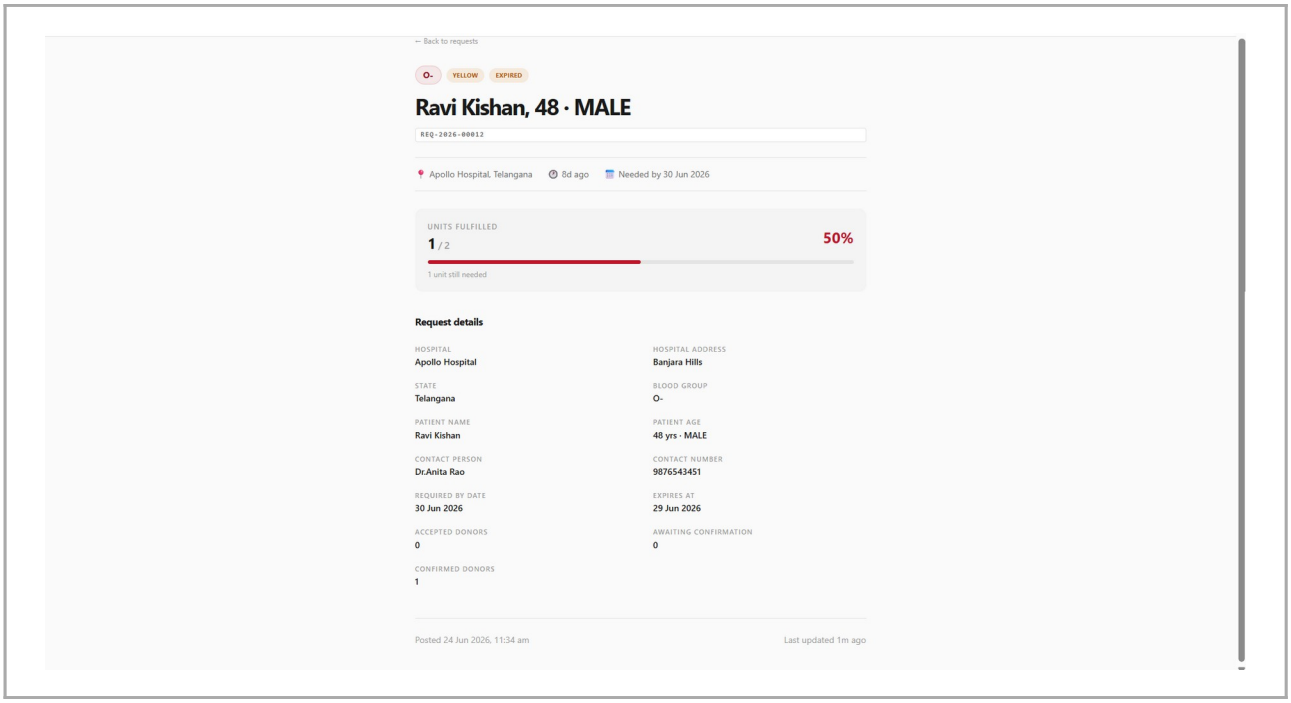


Figure 10: Donation History Page

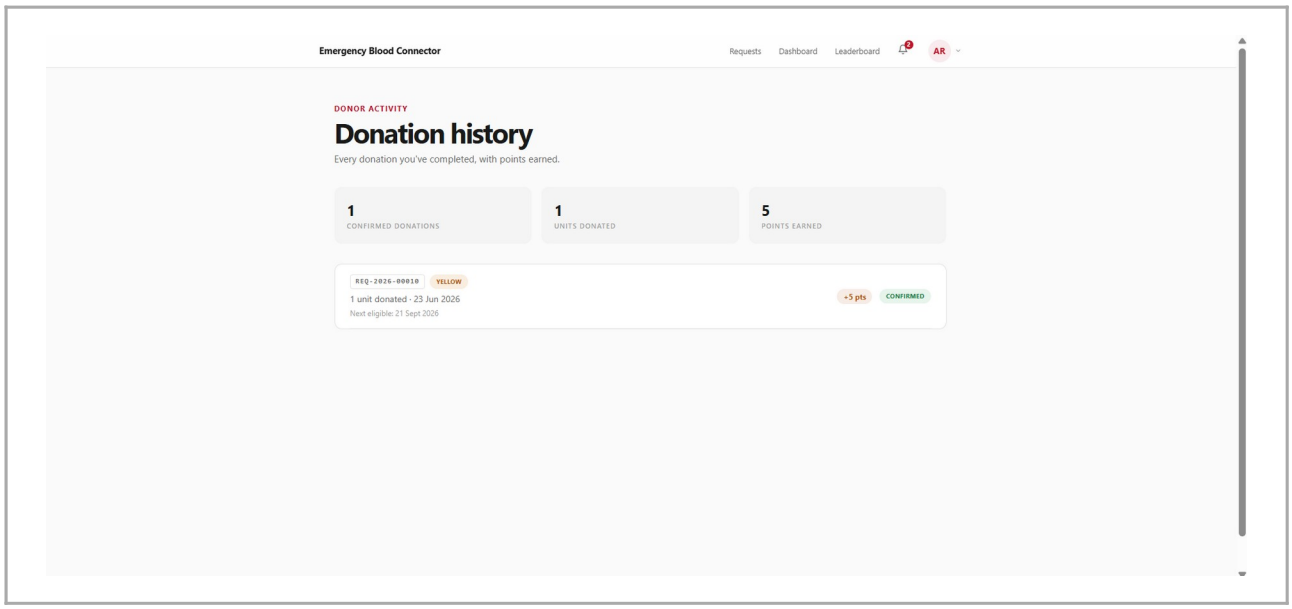


Figure 11: Badges Page

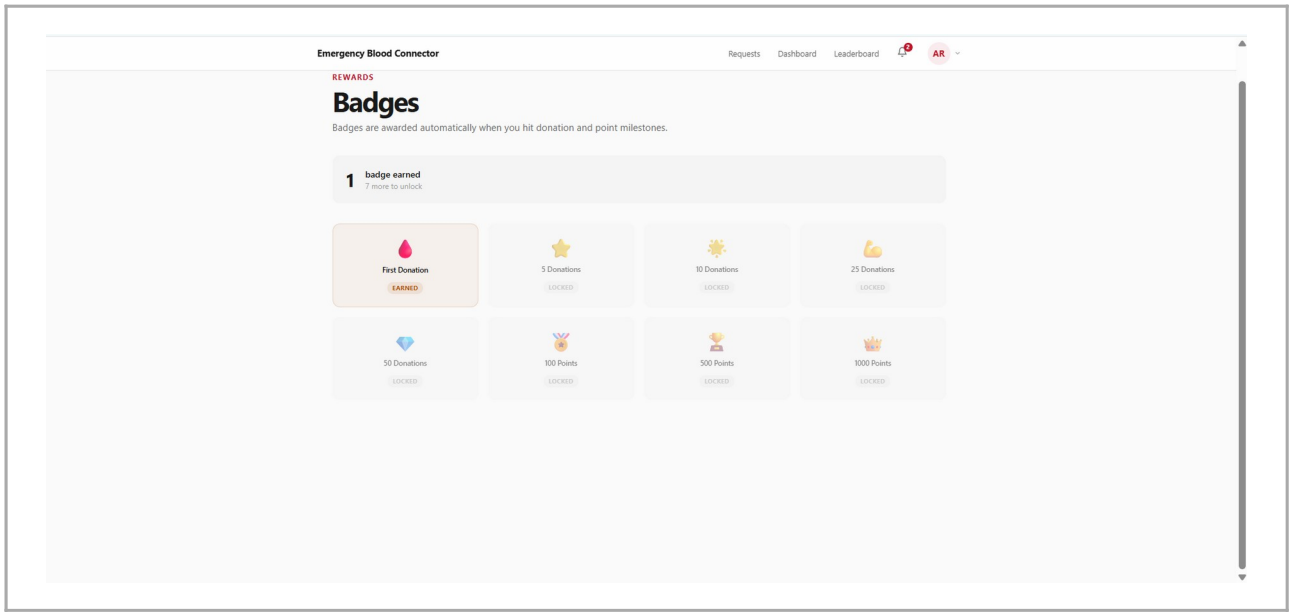


Figure 12: Achievements Page

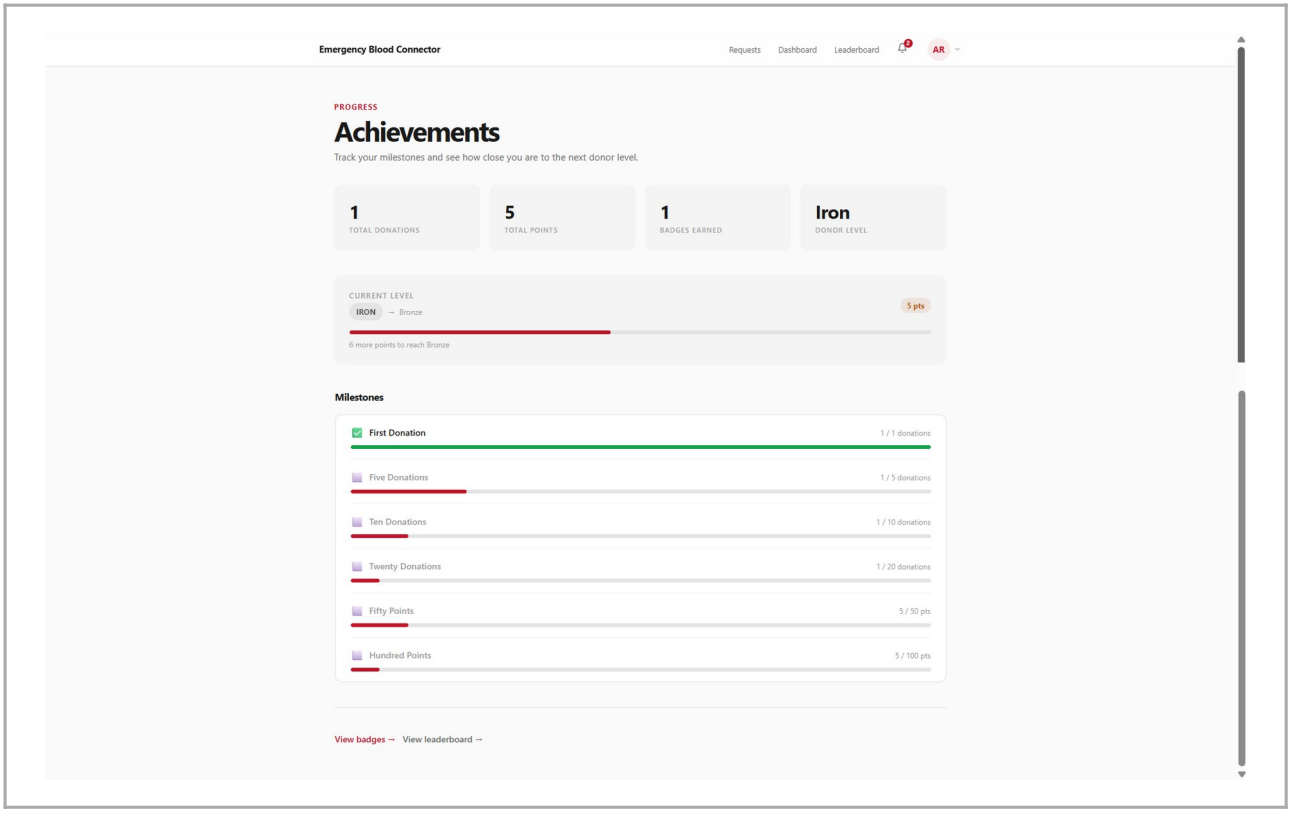


Figure 13: Leaderboard Page

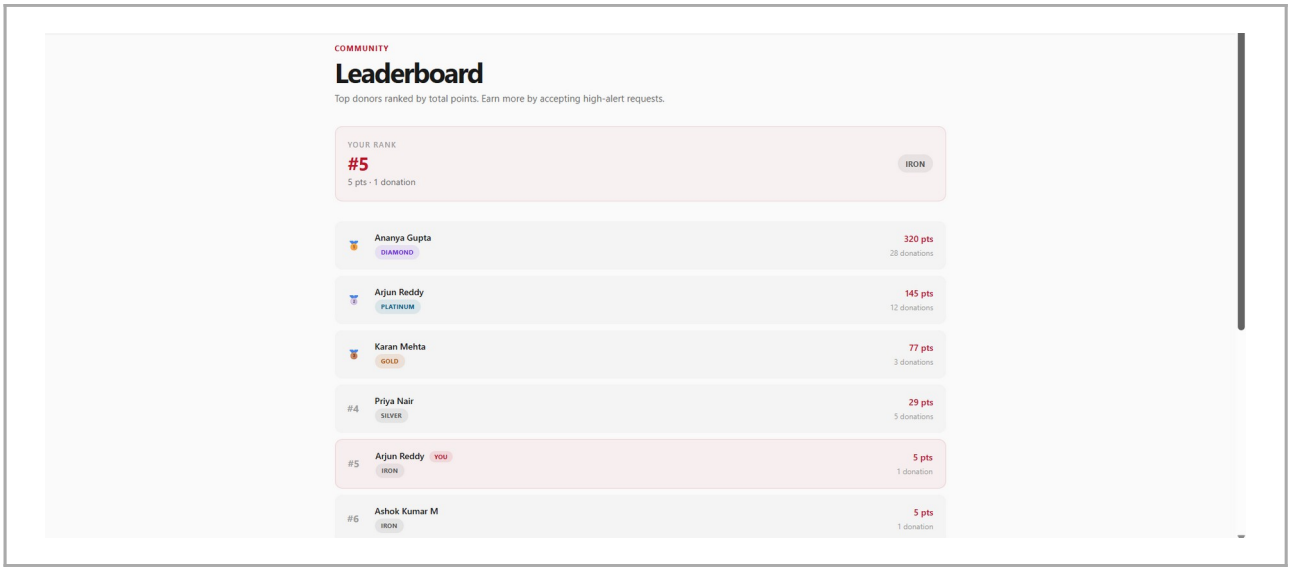


Figure 14: Notifications Page

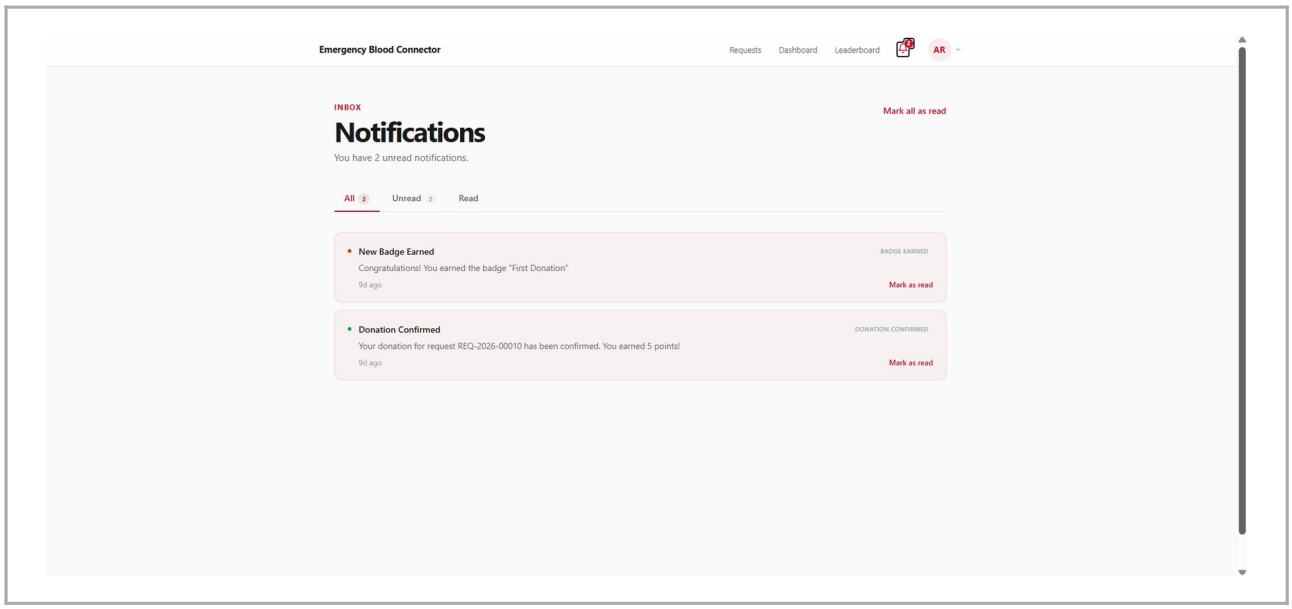


Figure 15: Admin Dashboard

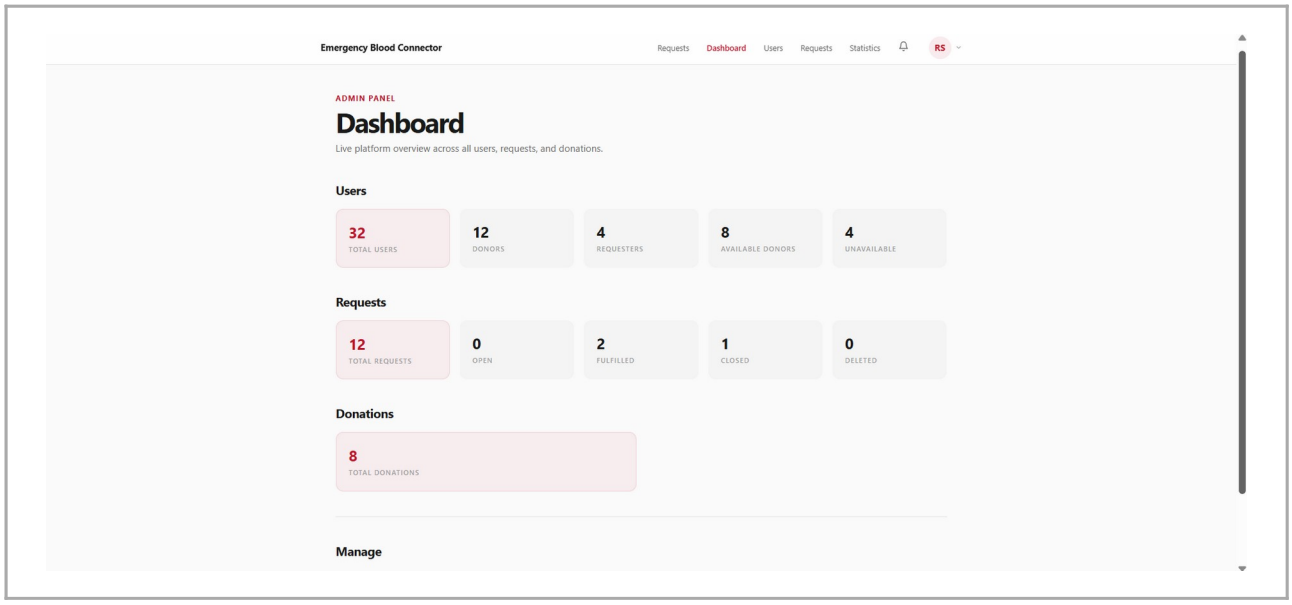


Figure 16: Admin User Management

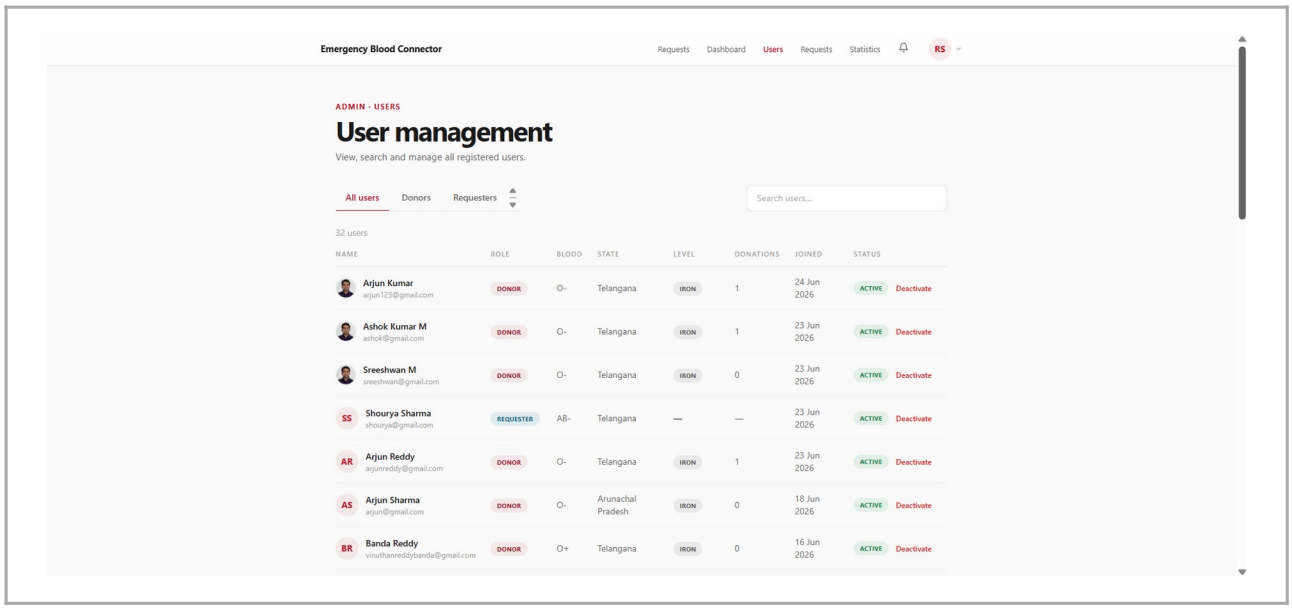


Figure 17: Admin Request Management

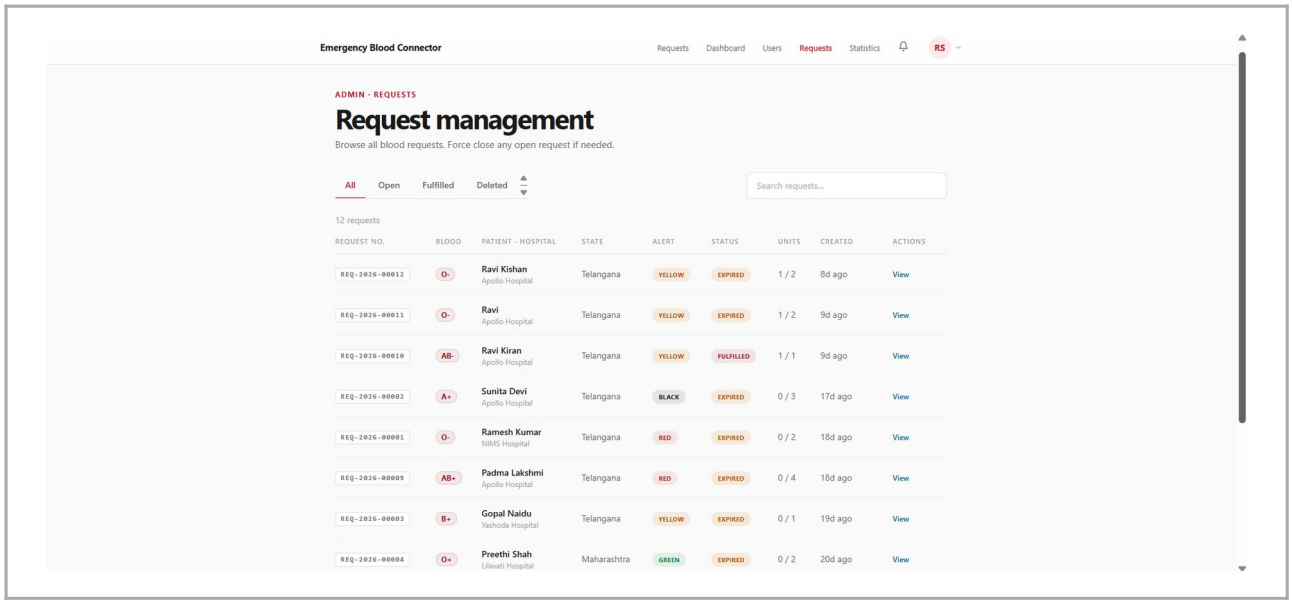
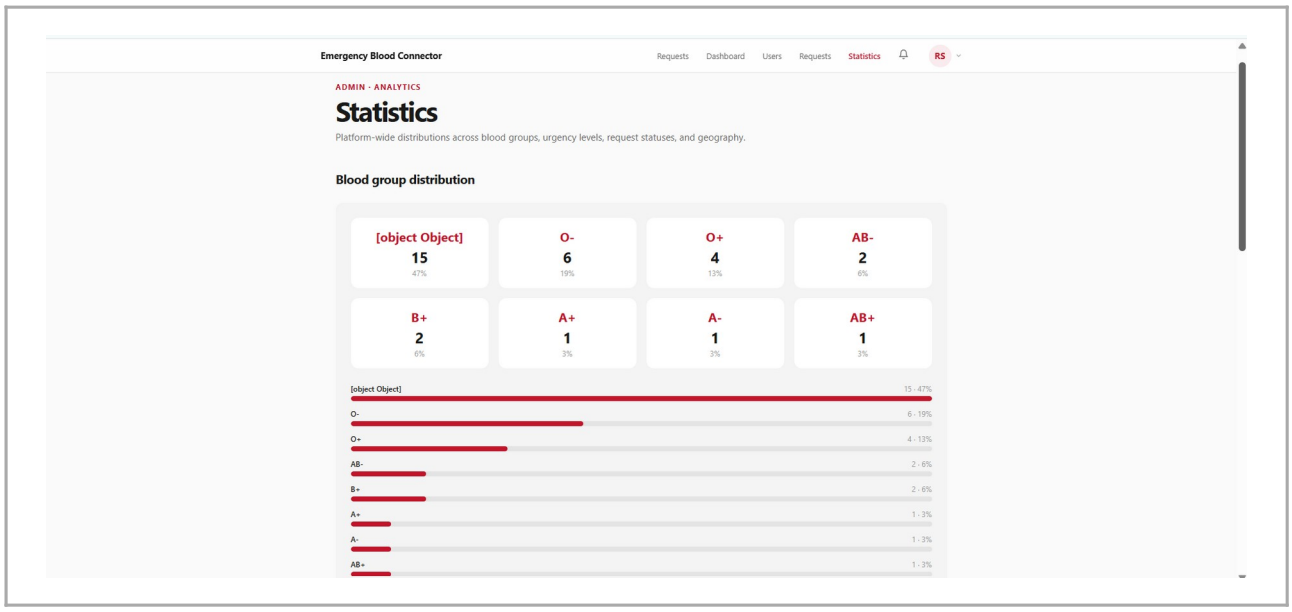


Figure 18: Admin Statistics (Blood Group / Alert Level Distribution)



ADVANTAGES AND LIMITATIONS

Advantages:

1. The platform is completely free and open, removing the cost and complexity barriers associated with commercial blood-management software.
2. Automatic blood-group compatibility filtering ensures donors only ever see requests they are genuinely eligible to fulfil, eliminating wasted outreach to incompatible donors.
3. The severity-based alert and priority scoring system ensures the most medically critical requests are surfaced first, without requiring any manual triage by staff.
4. The two-step verified donation workflow prevents both accidental double-booking of a donor across multiple requests and false claims of donation, since points and cooldown are only applied after independent requester confirmation.
5. The gamification layer — points, six-tier levels, milestone badges, and a public leaderboard — gives donors a lasting reason to return to the platform beyond a single emergency, addressing the repeat-donor retention problem that most existing solutions ignore entirely.
6. Automatic enforcement of the 90-day donation cooldown protects donor health by preventing a donor from being repeatedly matched before they are medically eligible to donate again.
7. Role-based access control (DONOR, REQUESTER, ADMIN) with JWT authentication and HTTP-only cookies keeps each user's capabilities scoped precisely to what their role requires.
8. The responsive Tailwind CSS interface works equally well on desktop and mobile, which matters for donors who are most likely to respond to a notification from their phone.
9. The administrative statistics dashboard gives visibility into blood-group shortages, alert-level distribution, and state-wise demand — insight that most peer-to-peer donor platforms do not surface at all.

Limitations:

1. Notifications are currently in-app only; there is no SMS, WhatsApp, or push notification channel, so a donor must actively open the application to see a new request.
2. There is no OTP-based mobile number verification during registration, so phone number authenticity is not independently confirmed at sign-up.

3. Password recovery currently requires the user to be already authenticated (change-password); there is no email-based forgot-password flow for a locked-out account.

4. The isHospitalVerified flag exists on the BloodRequestModel but there is no built-in workflow for a hospital or administrator to actually perform that verification yet.

5. The platform has no native mobile application; it is accessible only through a mobile-responsive web browser.

6. There is no automated test suite, so regression testing currently depends on manual verification during development.

7. Donor and requester matching is currently limited to state-level granularity rather than precise geolocation, so a donor and requester in different cities within the same state may still be matched despite being impractically far apart.

FUTURE ENHANCEMENTS

1. **Email and SMS Notifications:** Extending the existing in-app notification system with email and SMS delivery would ensure donors and requesters are alerted to critical events even when not actively using the application — particularly important for BLACK-alert emergencies.

2. **OTP-Based Verification:** Adding OTP verification of phone numbers during registration would strengthen identity assurance and reduce the risk of fraudulent donor or requester accounts.

3. **Email-Based Password Reset:** Implementing a forgot-password flow using a time-limited, emailed reset token would remove the current dependency on an already-authenticated session for password recovery.

4. **Hospital Verification Workflow:** Building an actual verification pipeline around the existing isHospitalVerified flag — allowing partner hospitals or administrators to confirm a request's legitimacy — would add a further trust layer for donors deciding whether to respond.

5. **Real-Time Notifications:** Replacing the current fetch-on-demand notification model with WebSocket-based push delivery would let donors see new compatible, high-priority requests the instant they are created.

6. **Native Mobile Application:** Developing companion Android and iOS applications using React Native would enable push notifications, offline caching of recent requests, and a more immediate response channel for time-critical alerts.

7. **Geolocation-Based Matching:** Extending the current state-level matching to precise geolocation (distance-based sorting) would let donors and requesters be matched by actual proximity rather than by state alone.

8. **Blood Bank / Hospital Integration:** Integrating with institutional blood bank inventories (in the spirit of platforms like eRaktKosh) would let the platform surface both individual donor matches and existing blood-bank stock in a single unified search.

CONCLUSION

The "Emergency Blood Connector" project successfully addresses a real and persistent gap in India's emergency healthcare ecosystem — the absence of a structured, verifiable, and prioritised digital platform connecting blood donors directly with patients in need. By combining automatic blood-group compatibility matching, a severity-based priority and expiry engine, and a two-step, independently verified donation workflow, the platform ensures that the most critical requests are surfaced first and that reward, cooldown, and trust mechanisms are only ever applied to donations that genuinely took place.

The gamification layer — points scaled to request urgency, six-tier donor levels, milestone badges, and a public leaderboard — directly targets the repeat-donor retention problem that existing solutions such as hospital registers, government portals, and voluntary donor databases largely leave unaddressed, giving donors a lasting, trackable reason to keep contributing beyond a single emergency.

Built on a modern, actively maintained technology stack — React 19, Express.js 5, and MongoDB on the backend and frontend — and deployed on Vercel and Render with MongoDB Atlas as the managed database layer, the application demonstrates the practical application of full-stack MERN development skills, role-based access control, and business-logic-driven system design to a genuinely high-stakes, real-world problem. With the future enhancements identified — particularly SMS/email alerting, hospital verification, and geolocation-based matching — the platform has a clear path toward becoming a more complete, production-grade solution for emergency blood coordination.

REFERENCES

1. React Documentation, "React — The library for web and native user interfaces," https://react.dev, 2026.
2. Express.js, "Express — Fast, unopinionated, minimalist web framework for Node.js," https://expressjs.com, 2026.
3. MongoDB, "MongoDB Documentation — The database for modern applications," https://www.mongodb.com/docs, 2026.
4. Mongoose, "Mongoose — Elegant MongoDB object modeling for Node.js," https://mongoosejs.com/docs, 2026.
5. Redux Toolkit, "The official, opinionated, batteries-included toolset for efficient Redux development," https://redux-toolkit.js.org, 2026.
6. React Hook Form, "Performant, flexible and extensible forms with easy-to-use validation," https://react-hook-form.com, 2026.
7. Tailwind CSS, "A utility-first CSS framework for rapid UI development," https://tailwindcss.com/docs, 2026.
8. Vite, "Next generation frontend tooling," https://vitejs.dev, 2026.
9. JSON Web Tokens, "JWT.IO — An open, industry standard RFC 7519 method for representing claims securely," https://jwt.io/introduction, 2026.
10. Cloudinary, "Cloudinary Documentation — Image and video API," https://cloudinary.com/documentation, 2026.
11. Vercel, "Vercel Documentation — Build and deploy the frontend cloud," https://vercel.com/docs, 2026.
12. Render, "Render Documentation — Cloud application hosting," https://render.com/docs, 2026.
13. eRaktKosh, "Centralized Blood Bank Management System," Ministry of Health and Family Welfare, Government of India, https://eraktkosh.mohfw.gov.in, 2026.
14. Friends2Support, "World's Largest Voluntary Blood Donors Database," https://www.friends2support.org, 2026.

SOURCE CODE

The complete source code of the Emergency Blood Connector project — including the backend (Express.js 5 / Mongoose 9) and frontend (React 19 / Vite 8) packages — is maintained in a public GitHub repository.

GITHUB REPOSITORY LINK —
https://github.com/SreeshwanMaralakshikari/Emergency_Blood_Connector

Live Deployment:

Frontend (Vercel): https://emergency-blood-connector-lilac.vercel.app

Backend API (Render): https://emergency-blood-connector-kbw2.onrender.com